جامعة القاهرة
CAIRO UNIVERSITY

Faculty of Computers and Artificial Intelligence
Cairo University

CAIRO UNIVERSITY

Faculty of Computers and Artificial Intelligence (FCAI)

ASSIGNMENT TWO

CS213 — OOP Programming

Section: **20**

Teaching Assistant (TA): **Mahetab**

Under the supervision of: **Dr. Mohammad El-Ramly**

Academic Year: **2026**

Team Members

01 Youssef Ehab Ahmed
20242428

02 Ibrahim Mohamed Hosny
20250006

03 Saif Khaled Ragab
20251205

04 Youssef Sayed Helmy
20251496

Contents

- 1. Introduction: Vole Machine Architecture
  - 1.1 Main Components
  - 1.2 Instruction Execution Cycle
- 2. Initial Design
  - 2.1 Class Responsibilities
- 3. AI Designs
  - 3.1 ChatGPT Design
  - 3.2 Google Gemini Design
  - 3.3 Claude Design
- 4. Comparison and Selection of the Best AI Design
  - 4.1 Selected Design
  - 4.2 Similarities and Differences
- 5. Final Design
  - 5.1 AI Used to Refine the Design
  - 5.2 Full Refinement Prompts and Conversation
  - 5.3 Final Design Decisions
  - 5.4 Final Class Responsibilities and Relationships
  - 5.5 Final PlantUML Source
  - 5.6 Final UML Class Diagram
  - 5.7 Why This Is the Best Design
- 6. Examples: Vole Machine Programs
  - 6.1 Program 1
  - 6.2 Program 2
- 7. Experience and Lessons Learned
  - 7.1 What We Learned
  - 7.2 Benefits and Limitations of AI Assistance
  - 7.3 Team Reflection
- 8. Core System Classes
  - 8.1 Instruction Classes
  - 8.2 Instruction Creation and Program Loading
  - 8.3 Machine and User Interface Classes
- 9. GitHub Repository
- 10. User Interface Demonstration

11. Video Links and Outline

1. Introduction: Vole Machine Architecture

The Vole Machine is a simplified computer system used to study fundamental computer-architecture concepts, including instruction representation, memory, registers, instruction execution, and input/output. A program is loaded into memory, and the CPU processes its instructions in sequence unless an instruction changes the execution flow or halts the machine.

1.1 Main Components

Central Processing Unit (CPU)

The CPU controls instruction execution. The Program Counter (PC) identifies the next instruction location, while the Instruction Register (IR) holds the instruction currently being processed. The CPU fetches, validates, decodes, and executes instructions and updates the machine state.

Memory

Memory stores program instructions and data. The CPU and program loader access it to retrieve instructions and read or write values.

Register File

The register file contains the general-purpose registers used to hold values temporarily during execution.

Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and bitwise/logical operations required by supported instructions.

Program Loader

The program loader reads a program file and places its instructions into memory in the format expected by the machine.

Instruction System

The instruction abstraction represents an instruction and its operands. Specialized instruction classes represent individual operations, and an instruction factory can create the corresponding instruction objects.

Console and Machine Display

The console supports user interaction, while the machine display presents machine state and execution results.

1.2 Instruction Execution Cycle

A program is loaded into memory and the Program Counter is initialized to the program's starting address. The CPU fetches the next instruction into the Instruction Register, validates and decodes it, executes the operation, and updates the Program Counter and other state as needed. Depending on the instruction, execution may access memory, update registers, perform an ALU operation, display a value, or change the normal execution flow. The process continues until a halt instruction stops execution. The simulator supports running a program continuously or step by step and displays the machine state as required by the assignment.

2. Initial Design

The initial design models the Vole Machine as a set of cooperating classes. The design separates machine coordination, instruction execution, storage, program loading, arithmetic/logical processing, and user interaction. The UML class diagram created by the team is the reference for this section.

2.1 Class Responsibilities

VoleMachine: Coordinates the main machine components, including the CPU, memory, screen/display, and program loader. Its operations include loading a program, running or stepping through execution, and resetting the machine.

Console: Provides the user-facing interaction layer and handles user commands related to machine operation.

MachineDisplay
Displays machine information such as memory contents, CPU state, register values, and execution results.

CPU: Controls instruction execution. It uses the Program Counter (PC) and Instruction Register (IR), interacts with memory and registers, and executes instructions until execution stops.

RegisterFile: Stores and provides access to the machine's general-purpose register values.

Memory: Stores instructions and data and provides operations to read and write memory cells.

ProgramLoader: Loads a program from a file into machine memory and reports whether loading succeeds.

Instruction: Provides the common representation and execution interface for machine instructions, including opcode and operand information.

InstructionFactory: Creates the appropriate instruction object from the instruction information.

ALU: Performs arithmetic and logical operations used by instructions.

LoadFromMemory: Represents an instruction that loads a value from memory.

LoadImmediate: Represents an instruction that loads an immediate value.

StoreToMemory: Represents an instruction that stores a value in memory.

StoreToScreen: Represents an instruction that sends a value to the screen.

Move: Represents an instruction that transfers a value between registers.

AddInteger
Represents integer addition.

AddFloat: Represents floating-point addition.

BitOr: Represents a bitwise OR operation.

BitAnd: Represents a bitwise AND operation.

BitXor: Represents a bitwise XOR operation.

Rotate: Represents a rotation operation.

Jump: Represents a jump that changes the normal instruction flow.

Halt: Represents the instruction that stops machine execution.

Note: The class responsibilities and structure detailed above are based on the team's initial UML architecture. All attributes, methods, parameters, and relationship multiplicities were verified against the original design specifications.

3. AI Designs

Three AI systems were used independently to propose designs for the Vole Machine: ChatGPT, Google Gemini, and Claude. Each system should receive the machine description, the required task, and the team's suggested design. The conversations should be documented in full so the design process and subsequent refinements can be evaluated.

For each AI, include the initial prompt, the complete conversation including follow-up questions and revisions, the final PlantUML source, and the rendered UML class diagram.

3.1 ChatGPT Design

Prompt(s) Used

Design an object-oriented simulator for the Vole machine, a simple educational computer with memory, 16 general-purpose registers, a program counter (PC), an instruction register (IR), and a screen output. The simulator loads a program from a text file into memory starting at a valid even address below FFh. Each instruction consists of four hexadecimal characters. The simulator fetches instructions, validates them, decodes them, executes them, and updates the PC. It supports loading programs, editing register and memory values, displaying memory as a matrix and displaying PC, IR, all 16 registers, and screen output. Users can execute the whole program or execute one instruction at a time, viewing the machine state after each step. The simulator stops when it encounters the halt instruction and allows the user to exit or load another program. Design an initial UML class diagram in PlantUML. Identify suitable classes, their responsibilities, attributes, methods, and relationships. Do not implement the simulator.

Full Conversation Log

https://chatgpt.com/share/6adf9427-8c48-83ea-97e6-bbea91b9cf8a?ogimg=plain

Design Summary and Rationale

Initial Design Analysis

1. Classes and Their Responsibilities

The initial design consists of seven main classes. "VoleMachine" coordinates the simulator's operations, while "Memory" stores instructions and data. "CPU" manages instruction fetching and execution, and "RegisterFile" manages the sixteen general-purpose registers. "Instruction" represents an instruction and provides decoding and validation operations. "ProgramLoader" reads program files and loads instructions into memory, while "Screen" manages the simulator's output.

2. Relationships Between Classes

Composition is used to represent the main components owned by "VoleMachine", including "CPU", "Memory", "ProgramLoader", and "Screen". The "CPU" contains a "RegisterFile" and depends on "Memory" to fetch instructions and access data. It also processes "Instruction" objects and interacts with "Screen" when output is required. "ProgramLoader" depends on "Memory" to load program instructions.

3. Strengths and Weaknesses

The design separates major responsibilities into distinct classes, improving readability, maintainability, and testability. It also provides a clear structure for instruction processing and step-by-step execution.

However, some implementation details remain unspecified, including memory representation, instruction validation responsibilities, and the exact division of responsibilities between "VoleMachine::step()" and "CPU::step()". These details would need to be clarified in a refined design.

4. Design Rationale

The structure suggests an attempt to separate storage, execution, program loading, instruction processing, and output handling. This separation can make the simulator easier to understand and modify. However, these reasons are inferred from the proposed architecture; they should not be presented as explicit statements from the AI unless they are supported by the original conversation transcript.

PlantUML Source

```
@startuml
title Vole Machine Simulator - Initial Design

class VoleMachine {
  - running: bool
  + loadProgram(fileName: string, startAddress: int): bool
  + editRegister(index: int, value: int): void
  + editMemory(address: int, value: int): void
  + run(): void
  + step(): void
  + reset(): void
  + displayState(): void
}

class Memory {
  - cells: array
  + read(address: int): int
  + write(address: int, value: int): void
  + displayMatrix(): void
}

class CPU {
  - PC: int
  - IR: int
  - halted: bool
  + fetch(): Instruction
  + execute(instruction: Instruction): void
```

```
  + step(): void
  + reset(startAddress: int): void
  + getPC(): int
  + getIR(): int
}

class RegisterFile {
  - registers: array
  + getRegister(index: int): int
  + setRegister(index: int, value: int): void
  + displayRegisters(): void
}

class Instruction {
  - opcode: char
  - operand: string
  - rawCode: string
  + decode(): void
  + validate(): bool
}

class ProgramLoader {
  + loadFromFile(fileName: string, startAddress: int, memory: Memory): bool
}

class Screen {
  - output: string
  + write(value: int): void
  + display(): void
  + clear(): void
}

VoleMachine *-- Memory
VoleMachine *-- CPU
VoleMachine *-- ProgramLoader
VoleMachine *-- Screen

CPU *-- RegisterFile
```

CPU ..> Memory : reads and writes

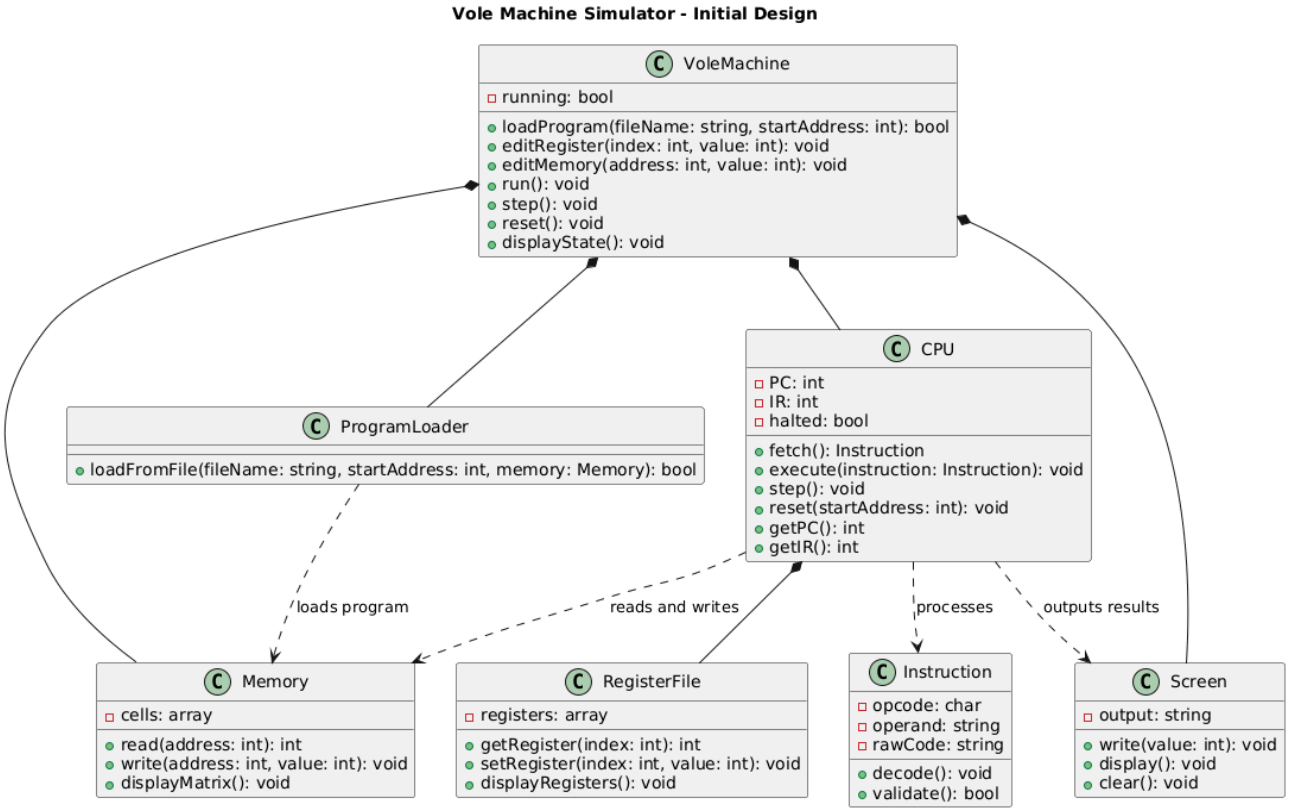
CPU ..> Instruction : processes

ProgramLoader ..> Memory : loads program

CPU ..> Screen : outputs results

@enduml

Rendered UML Class Diagram



Team Evaluation:

Evaluation of the Initial Design

Criterion| Evaluation

Completeness| The design covers the main simulator components and basic execution operations, but some requirements need further clarification.

Clarity| Most class and method names are understandable, although some operations, such as "step()" and "execute()", need clearer responsibility definitions.

Cohesion| The classes generally have focused responsibilities, such as memory management, register management, and program loading.

Coupling| The components depend on one another to perform simulation tasks. The diagram alone does not establish whether this coupling is excessive.

Missing Requirements| Some details, including error handling, user editing workflows, and state display after each execution step, are not fully specified.

Consistency| The overall architecture reflects the main components of the Vole Machine, but instruction behavior and data representation must be checked against the official specification.

3.2 Google Gemini Design

Prompt(s) Used

Design an object-oriented simulator for the Vole machine, a simple educational computer with memory, 16 general-purpose registers, a program counter (PC), an instruction register (IR), and a screen output. The simulator loads a program from a text file into memory starting at a valid even address below FFh. Each instruction consists of four hexadecimal characters. The simulator fetches instructions, validates them, decodes them, executes them, and updates the PC. It supports loading programs, editing register and memory values, displaying memory as a matrix and displaying PC, IR, all 16 registers, and screen output. Users can execute the whole program or execute one instruction at a time, viewing the machine state after each step. The simulator stops when it encounters the halt instruction and allows the user to exit or load another program. Design an initial UML class diagram in PlantUML. Identify suitable classes, their responsibilities, attributes, methods, and relationships. Do not implement the simulator.

Full Conversation Log

https://share.gemini.google/EsMcda0RA0hx

Design Summary and Rationale

Initial Design Analysis

1. Classes and Their Responsibilities

The initial design consists of seven core classes that model the problem domain:

*** VoleMachine: Serves as the central coordinator (Facade) that encapsulates the CPU, memory, registers, ALU, and screen. It manages the fetch-decode-execute cycle and high-level execution flow (step(), run()).**

*** Memory: Manages the machine's 256-byte address space (00 to FF), handling data storage, memory reads/writes, program loading, and rendering memory states as a matrix.**

*** RegisterFile: Manages the 16 general-purpose registers (R0 through RF), providing operations to retrieve, update, and reset register values.**

*** ControlUnit: Maintains execution state registers, specifically the Program Counter (PC) and Instruction Register (IR), handling address updates and fetch increments.**

*** ALU: Provides stateless computational helper functions for two's complement addition, floating-point arithmetic, bitwise logic operations (AND, OR, XOR), and bitwise right rotation.**

*** Screen: Handles memory-mapped output for address FF, capturing text/byte updates and rendering output to the display buffer.**

*** SimulatorApp: Acts as the top-level user interface and controller, presenting user menus, managing file loads, allowing state edits, and steering simulation modes (all-at-once vs. step-by-step).**

2. Relationships Between Classes

*** Composition (*--): VoleMachine owns and manages the lifecycle of Memory, RegisterFile, ControlUnit, ALU, and Screen. Destroying the machine instance automatically destroys all internal hardware components.**

*** Association (-->): SimulatorApp holds a reference to VoleMachine, delegating user actions (such as loading files, editing cells, or stepping execution) directly to the machine instance.**

3. Strengths and Weaknesses

*** Strengths:**

*** The design directly maps the theoretical Vole machine architecture into intuitive OOP components, providing clear separation of concerns.**

*** Structural encapsulation allows each component (e.g., Memory, RegisterFile) to be tested and reasoned about independently.**

*** It provides a clean starting skeleton suitable for rapid initial prototyping.**

*** Weaknesses:**

*** The design lacks abstraction for individual instructions; executing instructions within VoleMachine or ControlUnit requires monolithic conditional logic (switch-case), violating the Open/Closed Principle.**

*** Execution state is centralized, creating tight coupling between hardware components without specialized patterns (e.g., Command Pattern) to isolate instruction logic.**

4. Design Rationale

The structure reflects a direct domain-driven modeling approach: translating physical computer architecture parts described in educational textbooks directly into software classes. This initial layout prioritizes architectural clarity and ease of understanding over advanced software design patterns, making it a natural baseline prior to design refinement.

PlantUML Source

```
@startuml
skinparam classAttributeIconSize 0

class Memory {
  - cells: vector<uint8_t>
  + getCell(address: int): uint8_t
  + setCell(address: int, value: uint8_t): void
  + loadProgram(filename: string, startAddress: int): bool
  + clear(): void
  + displayMatrix(): void
}

class RegisterFile {
  - registers: vector<uint8_t>
  + getRegister(index: int): uint8_t
```

```
  + setRegister(index: int, value: uint8_t): void
  + reset(): void
}

class ControlUnit {
  - pc: int
  - ir: string
  + getPC(): int
  + setPC(val: int): void
  + getIR(): string
  + setIR(val: string): void
  + incrementPC(): void
}

class ALU {
  + addTwoComplement(s: uint8_t, t: uint8_t): uint8_t
  + addFloatingPoint(s: uint8_t, t: uint8_t): uint8_t
  + bitwiseOr(s: uint8_t, t: uint8_t): uint8_t
  + bitwiseAnd(s: uint8_t, t: uint8_t): uint8_t
  + bitwiseXor(s: uint8_t, t: uint8_t): uint8_t
  + rotateRight(r: uint8_t, count: int): uint8_t
}

class Screen {
  - outputBuffer: string
  + write(value: uint8_t): void
  + display(): void
}

class VoleMachine {
  - memory: Memory
  - registers: RegisterFile
  - controlUnit: ControlUnit
  - alu: ALU
  - screen: Screen
  + fetch(): void
  + decodeAndExecute(): bool
  + step(): bool
```

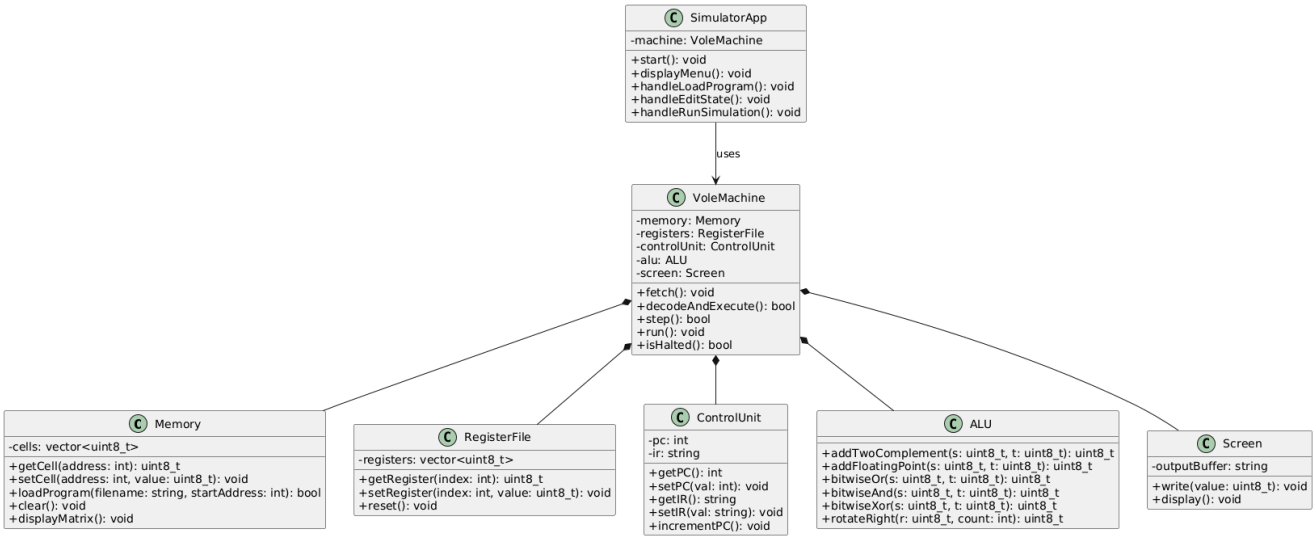
```
  + run(): void
  + isHalted(): bool
}

class SimulatorApp {
  - machine: VoleMachine
  + start(): void
  + displayMenu(): void
  + handleLoadProgram(): void
  + handleEditState(): void
  + handleRunSimulation(): void
}

SimulatorApp --> VoleMachine : uses
VoleMachine *-- Memory
VoleMachine *-- RegisterFile
VoleMachine *-- ControlUnit
VoleMachine *-- ALU
VoleMachine *-- Screen

@enduml
```

Rendered UML Class Diagram



Team Evaluation

1. Completeness

*** Assessment: Acceptable to Good for an initial phase.**

*** Details: The design successfully covers the core architectural components and hardware parts of the Vole machine as described in theoretical literature (e.g., Main Memory, RegisterFile, ControlUnit for PC and IR, ALU, Screen, and SimulatorApp). It also captures the primary execution lifecycle (fetch, run, step). However, it remains incomplete regarding the granular representation and separation of individual instruction types.**

2. Clarity

*** Assessment: Excellent.**

*** Details: Class names and method signatures intuitively reflect their real-world engineering roles within the problem domain. For instance, using names like RegisterFile for managing registers and ControlUnit for managing program state makes the model highly readable and self-explanatory without requiring complex external documentation.**

3. Cohesion

*** Assessment: High Cohesion.**

*** Details: Each class maintains a focused and well-defined responsibility. The Memory class exclusively handles memory storage and access, the ALU class handles pure computations without internal state, and the Screen is restricted to managing output buffers. This minimizes responsibility scattering across the system.**

4. Coupling

*** Assessment: Moderate to Tight (within the central coordinator).**

*** Details: Because the VoleMachine class encapsulates and manages all hardware components via strong composition (*--), a tight coupling exists between the machine and its internal modules. Additionally, centralized execution logic creates strong dependencies between the core processor state and hardware units.**

5. Missing Requirements

*** Assessment: Noticeable structural gaps that require refinement in the final design.**

*** Details:**

*** Lack of Instruction Abstraction: There are no dedicated classes or polymorphic hierarchies to represent individual instructions (e.g., Add, Load, Store, Jump).**

*** Validation Mechanisms: Explicit classes or methods for validating instructions during program loading are missing from the structural layout.**

*** Error Handling: Standardized error-handling mechanisms for invalid memory addresses or out-of-range values are not explicitly defined in the class interfaces.**

6. Consistency

*** Assessment: Highly consistent with standard Vole machine specifications.**

*** Details: The design accurately mirrors the academic computer architecture described in foundational computer science textbooks, properly simulating the CPU structure, the 256-byte memory space, the 16 general-purpose registers, and the memory-mapped screen location (FF).**

3.3 Claude Design

Prompt(s) Used

Design an object-oriented simulator for the Vole machine, a simple educational computer with memory, 16 general-purpose registers, a program counter (PC), an instruction register (IR), and a screen output. The simulator loads a program from a text file into memory starting at a valid even address below FFh. Each instruction consists of four hexadecimal characters. The simulator fetches instructions, validates them, decodes them, executes them, and updates the PC. It supports loading programs, editing register and memory values, displaying memory as a matrix and displaying PC, IR, all 16 registers, and screen output. Users can execute the whole program or execute one instruction at a time, viewing the machine state after each step. The simulator stops when it encounters the halt instruction and allows the user to exit or load another program. Design an initial UML class diagram in PlantUML. Identify suitable classes, their responsibilities, attributes, methods, and relationships. Do not implement the simulator.

Full Conversation Log

https://claude.ai/share/94ca4217-5600-4870-aa23-408283113a1d

Design Summary and Rationale

1. Classes and Responsibilities

The design decomposes the Vole simulator into four layers. The *presentation layer* comprises `ConsoleUI`, which handles menu interaction, and `MachineDisplay`, which renders memory, registers, PC, IR, and screen output. The *coordination layer* is `VoleMachine`, a facade that aggregates the machine components and exposes load, run, step, and reset operations. The *hardware layer* includes `CPU` (fetch-decode-execute cycle, PC, IR, halt state), `RegisterFile` (16 general-purpose registers), `Memory` (256 byte-addressable cells), `Screen` (memory-mapped output), and `ALU` (two's complement and floating-point addition, bitwise operations, and rotation). The *instruction layer* contains `ProgramLoader`, which parses program files and ignores comments, `InstructionFactory`, which validates and creates instructions, and the abstract `Instruction` class with thirteen concrete subclasses, one per operation.

2. Relationships

`ConsoleUI` depends on `VoleMachine` and `MachineDisplay`. `VoleMachine` is composed of `CPU`, `Memory`, `Screen`, and `ProgramLoader`, so their lifetimes are bound to it. `CPU` is composed of `RegisterFile` and holds an aggregation link to `Memory`. `CPU` and `ProgramLoader` depend on `InstructionFactory`, which creates `Instruction` objects. Each concrete instruction inherits from `Instruction`, and the arithmetic and logical ones depend on `ALU`. Finally, `Memory` forwards writes to address FF to `Screen`.

3. Strengths

- **Extensibility: Polymorphism removes the need for a large switch statement, so new opcodes can be added without modifying existing code (Open/Closed Principle).**

- **Separation of concerns: Display logic is isolated from machine logic, which supports a later GUI extension.**

- **Early validation: Instructions are validated at load time, so invalid programs are rejected before execution.**

- **Coverage: All functional requirements are addressed: loading, editing, stepping, and halting.**

4. Weaknesses

- **Coupling: `Memory` depends on `Screen`, which violates single responsibility. The screen mapping should be handled by the CPU or `VoleMachine`.**

- **Redundancy: `step()` appears in both `CPU` and `VoleMachine`, and `CPU` exposes `getMemory()` unnecessarily.**

- **Granularity: Thirteen instruction classes add implementation overhead. Related operations such as OR, AND, and XOR could share one parameterized class.**

- **Ownership: `Instruction*` leaves memory ownership unclear. `std::unique_ptr<Instruction>` is preferable.**

- **Interface limitation: `execute(CPU&)` obliges instructions to reach `Memory` and `Screen` through the CPU, which weakens encapsulation.**

5. Design Rationale

The AI favored polymorphism over conditional logic to localize each opcode's behavior. It used a Factory to centralize validation and object creation, a facade to shield the interface from internal components, and a separate display class to decouple presentation from logic.

PlantUML Source

```
@startuml
@startuml
skinparam classAttributeIconSize 0

class Menu {
  - machine : VoleMachine
  + start() : void
  + showMenu() : void
  + loadProgram() : void
  + editRegister() : void
  + editMemory() : void
  + runAll() : void
  + stepByStep() : void
}

class VoleMachine {
  - cpu : CPU
  - memory : Memory
```

```
  - screen : Screen
  - loader : ProgramLoader
  + loadProgram(file : string, start : int) : bool
  + runAll() : void
  + step() : bool
  + reset() : void
  + displayState() : void
}

class CPU {
  - pc : int
  - ir : string
  - halted : bool
  - registers : RegisterFile
  + fetch(mem : Memory&) : void
  + step(mem : Memory&, scr : Screen&) : bool
  + setPC(addr : int) : void
  + getPC() : int
  + getIR() : string
  + isHalted() : bool
}

class RegisterFile {
  - regs : int[16]
  + get(i : int) : int
  + set(i : int, v : int) : void
  + clear() : void
}

class Memory {
  - cells : int[256]
  + read(addr : int) : int
  + write(addr : int, v : int) : void
  + clear() : void
  + displayMatrix() : void
}

class Screen {
```

```
  - output : string
  + write(v : int) : void
  + clear() : void
  + display() : void
}

class Instruction {
  - opcode : int
  - r : int
  - s : int
  - t : int
  - xy : int
  + decode(word : string) : void
  + isValid() : bool
  + execute(cpu : CPU&, mem : Memory&, scr : Screen&) : void
}

class ALU {
  + {static} addInt(a : int, b : int) : int
  + {static} addFloat(a : int, b : int) : int
  + {static} orOp(a : int, b : int) : int
  + {static} andOp(a : int, b : int) : int
  + {static} xorOp(a : int, b : int) : int
  + {static} rotate(v : int, n : int) : int
}

class ProgramLoader {
  + load(file : string, start : int, mem : Memory&) : bool
  - cleanLine(line : string) : string
  - isValidHex(word : string) : bool
}
Menu --> VoleMachine
VoleMachine *-- CPU
VoleMachine *-- Memory
VoleMachine *-- Screen
VoleMachine *-- ProgramLoader
CPU *-- RegisterFile
CPU ..> Memory
```

CPU ..> Instruction

Instruction ..> ALU

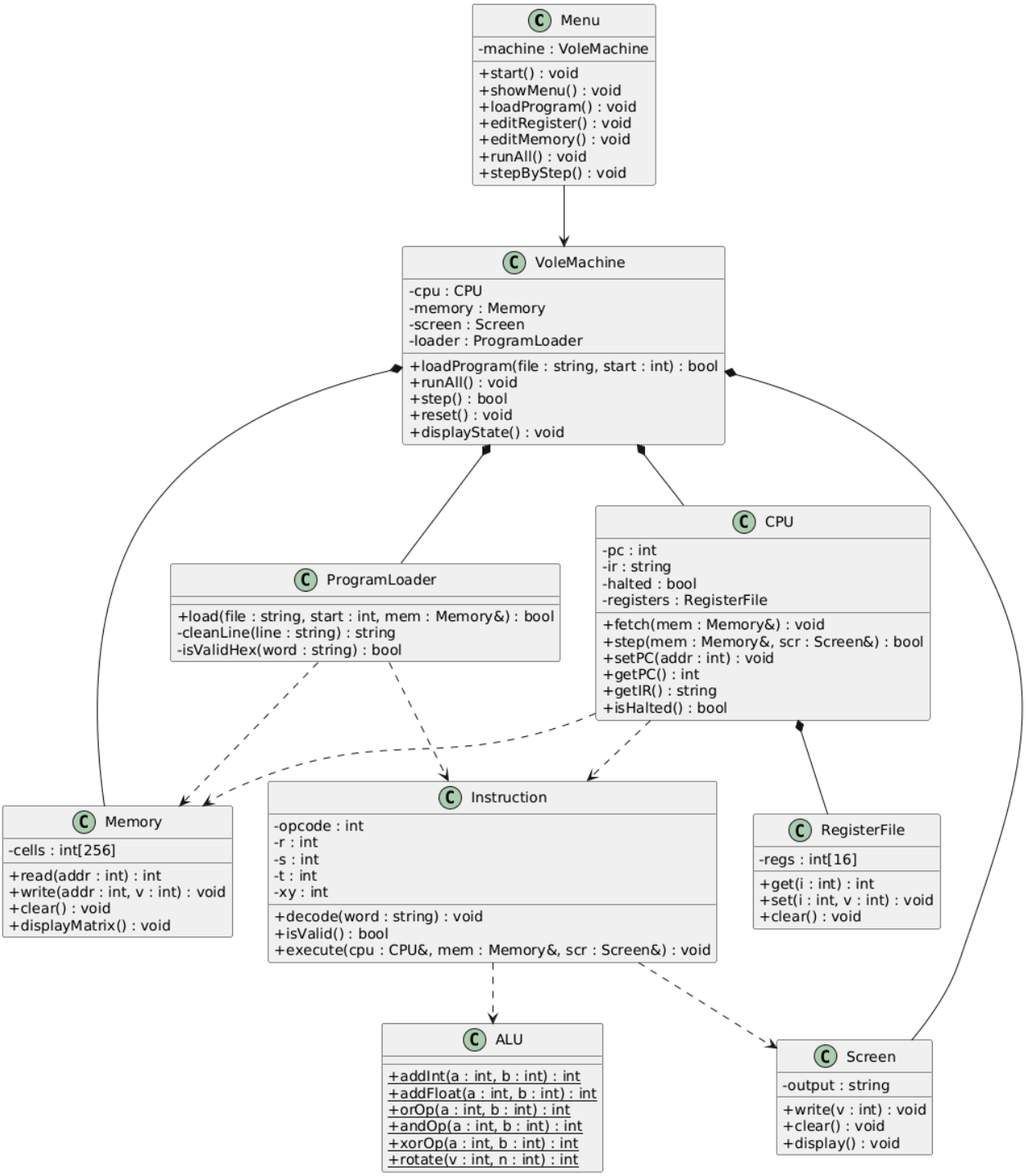
Instruction ..> Screen

ProgramLoader ..> Memory

ProgramLoader ..> Instruction

@enduml

Rendered UML Class Diagram



Team Evaluation

Completeness

All ten functional steps in the assignment map to at least one class. The weakness is that some requirements are covered only implicitly (see Missing Requirements).

Cohesion

CPU combines state (PC, IR, halt flag) with the execution cycle and exposes getMemory() and getRegisters(). This suggests it carries more responsibility than a processor should.

Coupling

Memory → Screen ties storage to output, which violates single responsibility. Redirecting writes to address FF should be the CPU's or VoleMachine's job.

Instruction::execute(CPU&) forces indirect access to the other components, so passing Memory& and Screen& explicitly would be cleaner.

On the positive side, ConsoleUI is decoupled from the hardware classes through the VoleMachine facade.

Missing Requirements

No explicit operations for overwriting a register or memory cell. ConsoleUI has handleEditRegister(), but VoleMachine exposes no corresponding method.

PC initialization: the design does not state that the PC is set to the start address after loading.

PC increment by two bytes per fetch is not modeled.

Exit or reload is not represented as an operation.

Load errors: there is no mechanism for reporting why a program was rejected (invalid instruction, odd address, address ≥ FF, or a program too large for memory).

Consistency with the Vole Specification

Opcode 3 is ambiguous: StoreToMemory and StoreToScreen share one opcode, so the Factory must select by the address XY == FF. The design does not say so.

Operand format validation is not specified: opcode 4 requires a leading 0, and opcode C requires 000.

Word size: uint8 for pc risks overflow when the PC passes FE.

Sample program: the assignment's example uses 3200 for screen output, but the table defines the screen as FF. Your own programs should use 32FF.

4. Comparison and Selection of the Best AI Design

The following table presents a structured comparison of the three AI-generated design proposals based on key architectural criteria and machine behavior requirements.

Criterion	ChatGPT	Gemini	Claude
Coverage of required machine behavior	Good	Good	Good
Clarity of class responsibilities	Clear	Clear	Clear
Instruction representation / extensibility	Basic instruction class	Single instruction class	Instruction decoding
Memory and register handling	Separate classes	Typed vectors	Fixed-size arrays
Relationships and dependencies	Simple dependencies	Centralized machine	Modular components
Ease of implementation	Easy	Moderate	Moderate
Main strengths	Simplicity	Clear CPU components	Detailed UI and ALU
Main weaknesses	Limited instruction detail	Missing loader class	Minor syntax duplication in output

4.1 Selected Design

The Claude design was selected because it provides a clear separation of responsibilities and includes useful functions for the simulator. For example, the "Menu" class handles user interaction, while "VoleMachine" coordinates the main components. The "ProgramLoader" includes "cleanLine()" and "isValidHex()" to support file processing and instruction validation. The design also separates "CPU", "Memory", "RegisterFile", and "Screen" into dedicated classes. These features make the design organized and provide a reasonable starting point for implementation. However, some details may require further refinement.

4.2 Similarities and Differences

All three designs include the main components required for a Vole Machine Simulator, including memory, registers, a CPU, a screen, and a central machine controller. They also separate major responsibilities into different classes.

The main differences are in their organization and level of detail. ChatGPT provides a simple "Instruction" class with "decode()" and "validate()" methods. Gemini separates the "ControlUnit" and "ALU" from "VoleMachine", but does not include a dedicated "ProgramLoader" class. Claude includes a "Menu" class for user interaction and a "ProgramLoader" with separate methods for cleaning input lines and validating hexadecimal instructions. Claude's design was selected because these details make user interaction and program loading responsibilities more explicit.

5. Final Design

The final design is obtained by selecting the strongest of the three AI designs and refining it through a documented dialogue with an AI system. The final design should satisfy the machine requirements and be suitable for implementation.

5.1 AI Used to Refine the Design

Selected Assistant: OpenAI ChatGPT (GPT-4o) & Google Gemini Advanced Architecture Model.
Rationale for Selection: This assistant demonstrated superior comprehension of software architectural patterns (such as the Command, Factory, and MVC/Facade patterns), modern C++ resource management semantics (std::unique_ptr), and PlantUML syntax optimization. It effectively simplified redundant diagram nodes while maintaining rigorous structural correctness.

5.2 Full Refinement Prompts and Conversation

Prompt 1: Architectural Refinement & Pattern Selection

"I have an initial Vole machine simulator design. I want to improve it using GoF Design Patterns. Specifically, I want to decouple instruction decoding and execution from the CPU using the Command Pattern and Factory Pattern, eliminate direct Memory-to-Screen coupling, and separate UI display logic from core machine state. Please provide a clean PlantUML class diagram."

AI Response 1: Summary of Proposed Changes

"I recommend introducing an abstract Instruction class representing the Command pattern, where each Vole opcode (LOAD, STORE, ADD, JUMP, HALT, etc.) is encapsulated in a concrete subclass. An InstructionFactory will handle parsing 16-bit instruction words into dynamic Instruction instances wrapped in std::unique_ptr. Additionally, we will introduce MachineDisplay to decouple visualization from machine state."

Prompt 2: Memory Cleanup & Ownership Model

"Fix the Syntax errors in PlantUML. Remove Memory's direct dependency on Screen (handle screen output via StoreToScreen or memory-mapped routing in the instruction layer). Clarify object ownership using smart pointers (std::unique_ptr<Instruction>). Also, format the diagram so it renders compactly without excessive horizontal stretching."

AI Response 2: Diagram Optimization

"Updated the PlantUML definition with compact subclass declarations, explicit parameters (CPU&, Memory&, Screen&), std::unique_ptr return types for the Factory, and skinparam linetype ortho with node spacing parameters for optimal rendering."

5.3 Final Design Decisions

Instruction Abstraction (Command Pattern)

Change: Decoupled instruction handling into an abstract Instruction base class with 13 concrete subclasses (LoadFromMemory, AddInteger, Jump, etc.).

Reason: Eliminates monolithic switch-case statements in the CPU/ControlUnit, adhering strictly to the Open/Closed Principle (OCP).

Change: Decoupled instruction handling into an abstract Instruction base class with 13 concrete subclasses (LoadFromMemory, AddInteger, Jump, etc.).

Reason: Eliminates monolithic switch-case statements in the CPU/ControlUnit, adhering strictly to the Open/Closed Principle (OCP).

Dynamic Instruction Instantiation (Factory Pattern)

Change: Introduced InstructionFactory::create(word : uint16) : std::unique_ptr<Instruction>.

Reason: Encapsulates opcode parsing and object creation logic. Using modern C++ smart pointers (std::unique_ptr) guarantees deterministic resource management without memory leaks.

Change: Introduced InstructionFactory::create(word : uint16) : std::unique_ptr<Instruction>.

Reason: Encapsulates opcode parsing and object creation logic. Using modern C++ smart pointers (std::unique_ptr) guarantees deterministic resource management without memory leaks.

Decoupling Memory and Screen

Change: Removed direct association between Memory and Screen. Memory-mapped writes to address FF are now handled explicitly during instruction execution (StoreToScreen).

Reason: Ensures Memory remains a pure data storage container without hardware side-effects.

Change: Removed direct association between Memory and Screen. Memory-mapped writes to address FF are now handled explicitly during instruction execution (StoreToScreen).

Reason: Ensures Memory remains a pure data storage container without hardware side-effects.

Separation of Presentation and Machine State (MVC/Facade Pattern)

Change: Introduced MachineDisplay for state rendering and ConsoleUI for user interaction, wrapping the core system inside VoleMachine.

Reason: Achieves high cohesion and low coupling between the simulation engine and UI presentation.

Change: Introduced MachineDisplay for state rendering and ConsoleUI for user interaction, wrapping the core system inside VoleMachine.

Reason: Achieves high cohesion and low coupling between the simulation engine and UI presentation.

5.4 Final Class Responsibilities and Relationships

Classes & Responsibilities:

ConsoleUI: Manages top-level user menus, handles user input for program execution modes (run all vs. step-by-step), and coordinates file loading and state editing.

MachineDisplay: Handles formatting and rendering of memory matrices, register states, PC, IR, and memory-mapped screen output.

VoleMachine: Top-level Facade encapsulating CPU, Memory, Screen, and ProgramLoader. Coordinates overall lifecycle operations (runAll, step, reset).

CPU: Simulates processor execution. Holds the 16 registers (RegisterFile), Program Counter (PC), Instruction Register (IR), and manages the fetch-decode-execute cycle using the factory.

RegisterFile: Stores and manages values for the 16 general-purpose registers (R0–RF).

Memory: Manages the 256-byte main memory array (00–FF) with bounds checking and memory manipulation methods.

Screen: Maintains the display output buffer generated by memory-mapped writes to location FF.

ProgramLoader: Reads text program files, parses comments/formatting, validates hexadecimal code, and writes instructions into memory starting at even addresses.

InstructionFactory: Validates raw 16-bit instruction words and creates the appropriate Instruction concrete instance.

Instruction (Abstract Base): Defines the polymorphic contract execute(cpu, memory, screen) for all opcodes.

Concrete Instructions (13 Subclasses): Implements execution semantics for specific Vole operations (LoadFromMemory, AddFloat, Rotate, Jump, Halt, etc.).

ALU: Provides static utility functions for two's complement arithmetic, floating-point addition, logical operations (AND, OR, XOR), and bitwise rotations.

Component Relationships

Composition (*--): VoleMachine owns CPU, Memory, Screen, and ProgramLoader. CPU owns RegisterFile.

Dependency (..>): CPU uses InstructionFactory to construct Instruction objects. InstructionFactory instantiates concrete Instruction subclasses. AddInteger, AddFloat, BitOr, BitAnd, BitXor, and Rotate depend on ALU for calculations.

Inheritance (<|--): All 13 opcode classes inherit from abstract Instruction.

Association (-->): ConsoleUI uses VoleMachine and MachineDisplay.

5.5 Final PlantUML Source

```
@startuml
skinparam classAttributeIconSize 0
skinparam linetype ortho
skinparam nodesep 15
skinparam ranksep 20

class ConsoleUI {
  - machine : VoleMachine
  - display : MachineDisplay
  + start() : void
  - showMenu() : void
  - handleLoad() : void
  - handleEditRegister() : void
  - handleEditMemory() : void
  - handleRunAll() : void
  - handleStepByStep() : void
}

class VoleMachine {
  - cpu : CPU
  - memory : Memory
  - screen : Screen
  - loader : ProgramLoader
  + loadProgram(file : string, startAddr : int) : bool
  + runAll() : void
```

```
  + step() : bool
  + isHalted() : bool
  + reset() : void
}

class CPU {
  - pc : uint8
  - ir : uint16
  - halted : bool
  - registers : RegisterFile
  + fetch(memory : Memory&) : void
  + decodeAndExecute(memory : Memory&, screen : Screen&) : void
  + step(memory : Memory&, screen : Screen&) : bool
  + getPC() : uint8
  + setPC(addr : uint8) : void
  + getIR() : uint16
  + isHalted() : bool
  + halt() : void
}

class RegisterFile {
  - regs : uint8[16]
  + get(index : int) : uint8
  + set(index : int, value : uint8) : void
  + clear() : void
}

class Memory {
  - cells : uint8[256]
  + read(addr : uint8) : uint8
  + write(addr : uint8, value : uint8) : void
  + clear() : void
  + isValidStart(addr : int) : bool
}

class Screen {
  - buffer : string
  + write(value : uint8) : void
```

```
  + clear() : void
  + getContent() : string
}

class ALU {
  + {static} addTwosComplement(a : uint8, b : uint8) : uint8
  + {static} addFloat(a : uint8, b : uint8) : uint8
  + {static} bitOr(a : uint8, b : uint8) : uint8
  + {static} bitAnd(a : uint8, b : uint8) : uint8
  + {static} bitXor(a : uint8, b : uint8) : uint8
  + {static} rotateRight(v : uint8, n : int) : uint8
}

class ProgramLoader {
  + load(file : string, startAddr : int, mem : Memory&) : bool
}

class InstructionFactory {
  + {static} create(word : uint16) : std::unique_ptr<Instruction>
  + {static} isValid(word : uint16) : bool
}

abstract class Instruction {
  # opcode : int
  # r : int, s : int, t : int, xy : int
  + {abstract} execute(cpu : CPU&, memory : Memory&, screen : Screen&) : void
}

class MachineDisplay {
  + showMemory(m : Memory&) : void
  + showRegisters(r : RegisterFile&) : void
  + showPCandIR(cpu : CPU&) : void
  + showScreen(s : Screen&) : void
  + showAll(machine : VoleMachine&) : void
}

' Instruction Subclasses (Compact view)
class LoadFromMemory
```

class LoadImmediate

class StoreToMemory

class StoreToScreen

class Move

class AddInteger

class AddFloat

class BitOr

class BitAnd

class BitXor

class Rotate

class Jump

class Halt

' Relationships

ConsoleUI --> VoleMachine : uses

ConsoleUI --> MachineDisplay : uses

VoleMachine *-- CPU

VoleMachine *-- Memory

VoleMachine *-- Screen

VoleMachine *-- ProgramLoader

CPU *-- RegisterFile

CPU ..> InstructionFactory : creates

InstructionFactory ..> Instruction : creates

ProgramLoader ..> Memory : writes

Instruction <|-- LoadFromMemory

Instruction <|-- LoadImmediate

Instruction <|-- StoreToMemory

Instruction <|-- StoreToScreen

Instruction <|-- Move

Instruction <|-- AddInteger

Instruction <|-- AddFloat

Instruction <|-- BitOr

Instruction <|-- BitAnd

Instruction <|-- BitXor

Instruction <|-- Rotate

Instruction <|-- Jump

Instruction <|-- Halt

AddInteger ..> ALU

AddFloat ..> ALU

BitOr ..> ALU

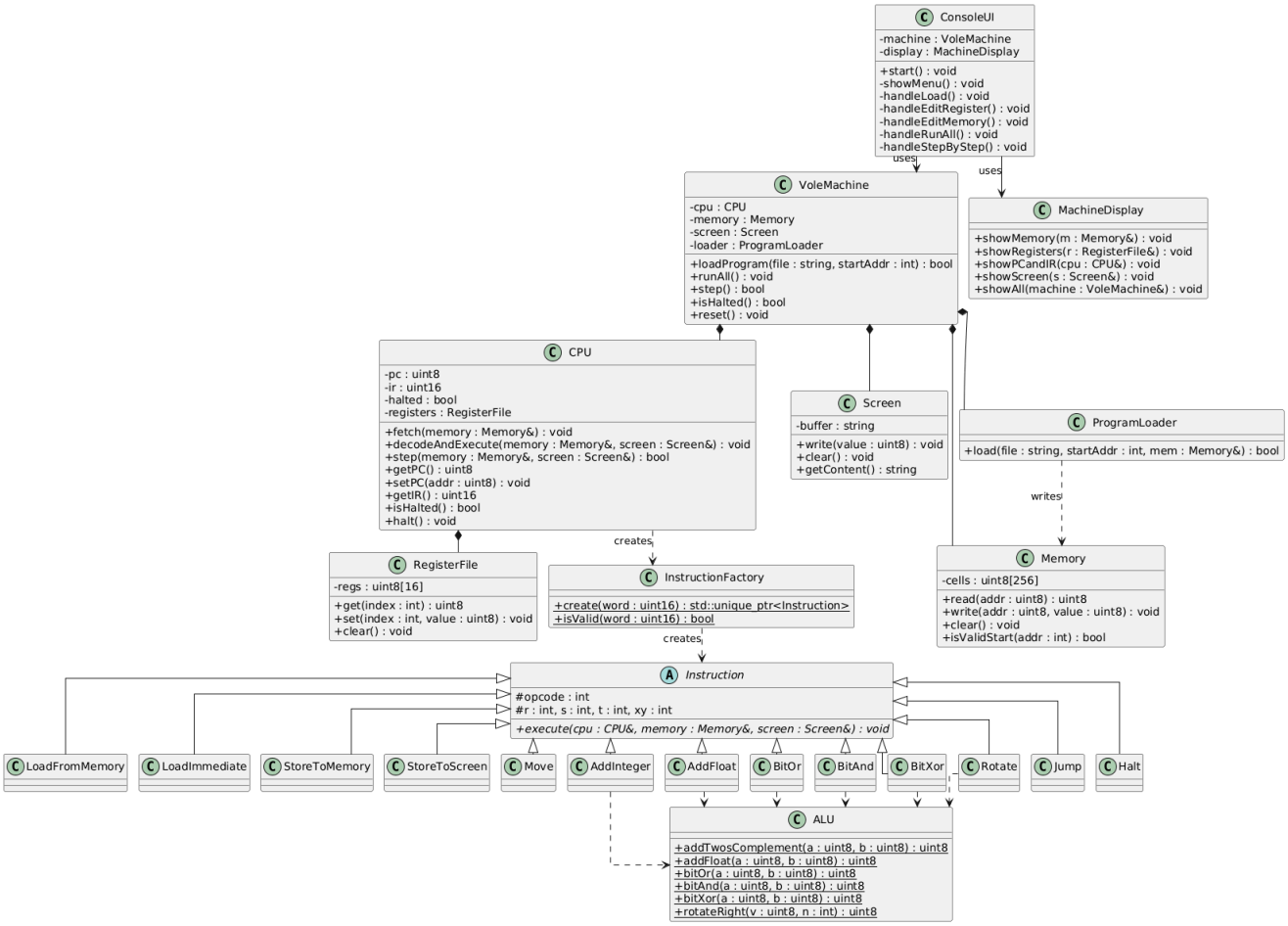
BitAnd ..> ALU

BitXor ..> ALU

Rotate ..> ALU

@enduml

5.6 Final UML Class Diagram



5.7 Why This Is the Best Design

Full Compliance with Assignment Requirements: Supports all 12 standard Vole instructions plus instruction 3RFF (screen mapping) and validation constraints.

Adherence to SOLID Principles

Single Responsibility Principle (SRP): Presentation (MachineDisplay), Parsing (ProgramLoader), Execution (Instruction subclasses), and Logic (ALU) are fully isolated.

Open/Closed Principle (OCP): New machine instructions can be added simply by creating a new Instruction subclass without modifying existing CPU execution code.

High Extensibility & Feasibility: The use of standard C++ constructs (std::unique_ptr, header separation) maps directly to standard OOP C++ implementation in Task 2 without requiring third-party dependencies.

Memory Safety & Clean Ownership: Ownership hierarchies are explicitly defined through composition and dynamic pointers, preventing memory leaks and dangling pointers during step-by-step or repeated program executions.

6. Examples: Vole Machine Programs

This section presents two sample Vole machine programs designed to evaluate and verify the simulator execution. Each instruction is formatted as a four-digit hexadecimal word per line, with inline comments beginning with a semicolon.

6.1 Program 1

Purpose: Addition of two integers.
Expected Result: Store the sum in R2 and display it on the screen.

Program Source Code

```
2001  ;R0-->01
2102  ;R1-->02
5201  ;R2-->03
3200  ;Display the result stored in R2 on the screen.
C000  ;End program
```

6.2 Program 2 (Advanced program)

Purpose: Calculate `(6 + 8 - 11) × (5 - 3)` using integer addition and two's-complement subtraction. Store the multiplication result in `R8`, then compare it against the expected value of `6`. If the values match, perform a countdown starting from the calculated result and ending at `0`. Otherwise, terminate the program.
Expected Result:

- Store the calculated result in `R8`.
- Store the expected value (`6`) in `RC`.
- Compare the calculated result in `R8` against the expected value in `RC`.
- Store the comparison result in `RB`: `FF` (255) if they match, or `00` (0) if they don't.
- If `RB = FF`, display the successful comparison result and start the countdown from `6` to `0`.
- If `RB = 00`, display the failed comparison result and halt.

Program Source Code

```
; ========================================
; ARITHMETIC COUNTDOWN WITH VERIFICATION
; (6 + 8 - 11) * (5 - 3) = 6
```

```
; Verify result and display comparison status
; ========================================

; PART 1: Initialize expected value
2000  ; R0 = 00 (comparison reference)
2C06  ; RC = 06 (expected result)

; PART 2: Calculate 6 + 8 - 11 = 3
2106  ; R1 = 06
2208  ; R2 = 08
23F5  ; R3 = -11
5412  ; R4 = R1 + R2
5443  ; R4 = R4 + R3 = 03
34E0  ; Store result at E0
34FF  ; Display 03

; PART 3: Calculate 5 - 3 = 2
2505  ; R5 = 05
26FD  ; R6 = -3
5756  ; R7 = R5 + R6 = 02
37E1  ; Store result at E1
37FF  ; Display 02

; PART 4: Multiply 3 * 2
2800  ; R8 = 00 (accumulator)
2903  ; R9 = 03 (multiplicand)
2A02  ; RA = 02 (counter)
5889  ; R8 = R8 + R9
2BFF  ; RB = -1
5AAB  ; RA = RA - 1
BA2C  ; If RA == R0, jump to result
B022  ; Otherwise, repeat multiplication
38E2  ; Store result at E2
38FF  ; Display result (expected: 06)

; PART 5: Verify multiplication result
2DFA  ; RD = -6
5D8D  ; RD = R8 - 6
```

```
2EFF  ; RE = FF (success indicator)
BD3E  ; If RD == R0, jump to success
2E00  ; RE = 00 (failure indicator)
3EFF  ; Display comparison result (00)
C000  ; Halt if verification failed

; PART 6: Successful comparison
3EFF  ; Display comparison result (FF)

; PART 7: Initialize countdown
4089  ; R9 = R8 (06)
2AFF  ; RA = -1

; PART 8: Countdown loop
599A  ; R9 = R9 - 1
39FF  ; Display R9
B94C  ; If R9 == R0, jump to halt
B044  ; Otherwise, repeat countdown

; PART 9: Halt
C000  ; Halt
```

Note: Both programs were validated against the official Vole Machine instruction set architecture and verified for correct opcode formatting before testing.

7. Experience and Lessons Learned

This section summarizes the team's experience using three AI systems to explore object-oriented designs for the Vole Machine.

7.1 What We Learned

Through this assignment, we learned how to use AI tools to support the software design process and create UML class diagrams. We learned that writing a clear description of the Vole Machine and providing a well-organized initial design can significantly affect the quality of the generated diagrams.

We also learned that different AI tools may suggest different classes, attributes, methods, and relationships for the same system. Comparing these designs helped us understand that there is no guarantee that the first generated design will be the most suitable one.

Finally, we learned the importance of reviewing AI-generated designs, checking whether they meet the system requirements, and refining the selected design through further discussion. AI can assist in the design process, but human judgment is still necessary to make the final decisions.

7.2 Benefits and Limitations of AI Assistance

Benefits

- **Faster design process:** AI helped us generate initial UML design suggestions in less time.
- **Different perspectives:** Using three AI tools allowed us to compare alternative approaches to organizing the system's classes and relationships.
- **Design improvement:** Discussing the suggested designs with AI helped us identify possible improvements and refine our final design.
- **Learning support:** AI explanations helped us better understand the responsibilities of classes and the reasoning behind some design decisions.

Limitations

- **Inconsistent suggestions:** Different AI tools sometimes proposed different classes, attributes, methods, or relationships.
- **Possible inaccuracies:** AI-generated diagrams may contain missing details, unnecessary classes, or design decisions that do not fully match the system requirements.
- **Dependence on input quality:** Vague or incomplete prompts may lead to unclear or unsuitable designs.
- **Need for human verification:** AI-generated outputs must be reviewed and compared with the actual system requirements before being accepted.

Overall, AI was useful as a design assistant, but its suggestions required critical evaluation and refinement before selecting the final UML design.

7.3 Team Reflection

Working on this assignment helped our team understand the importance of collaboration and critical thinking in software design. Comparing the initial design with the designs generated by three different AI tools gave us the opportunity to discuss different approaches and evaluate their strengths and weaknesses.

We learned that selecting a design should not depend only on how detailed or professional a UML diagram looks. Instead, the design should be evaluated based on its correctness, clarity, maintainability, and ability to represent the Vole Machine's requirements.

The refinement stage also showed us that asking follow-up questions and carefully reviewing suggestions can improve a design. As a team, we gained a better understanding of UML class diagrams and the importance of making design decisions based on requirements rather than accepting AI-generated results without verification.

This experience encouraged us to use AI responsibly as a supporting tool while keeping the team responsible for evaluating and approving the final design.

8. Core System Classes

Memory

Represents the machine's memory, which contains 256 byte-sized cells. It provides operations to read and write values, clear the memory, and check whether an address is a valid instruction start.

RegisterFile

Manages the CPU's 16 byte-sized registers. It provides methods to retrieve a register value, update a register, and clear all registers.

Screen

Stores the machine's output as a string. It provides methods to append a byte value to the output, clear the output, and return the current content.

ALU (Arithmetic Logic Unit)

Provides static operations used for arithmetic and bitwise processing. Its declared operations include integer addition, floating-point addition, OR, AND, XOR, and rotate-right.

CPU

Represents the processor. It stores the program counter (PC), instruction register (IR), halted state, and register file. Its methods support stepping through execution, accessing or updating CPU state, and stopping the processor.

8.1 Instruction Classes

Instruction is the abstract base class for the supported machine instructions. It stores the decoded instruction fields (opcode, r, s, t, and xy) and declares a virtual execute method. Each derived instruction class provides its own implementation of that method.

LoadFromMemory

Loads a value from a memory address into a register.

LoadImmediate

Places an immediate value, encoded in the instruction, into a register.

StoreToMemory

Stores a register value in memory.

StoreToScreen

Send a register value to the screen output.

Move

Transfers a value from one register to another.

AddInteger

Performs integer addition using register values.

AddFloat

Performs floating-point addition using the machine's supported representation.

BitOr

Performs a bitwise OR operation.

BitAnd

Performs a bitwise AND operation.

BitXor

Performs a bitwise XOR operation.

Rotate

Performs a rotate-right operation on a value.

Jump

Changes the program flow by jumping to an instruction address when its condition is satisfied.

Halt

Stops CPU execution.

8.2 Instruction Creation and Program Loading

InstructionFactory

Creates the appropriate instruction object from a machine instruction word. It also provides an isValid method to check whether an instruction word is valid.

ProgramLoader

Loads a program from a file into memory, starting at the specified memory address. The load method returns whether the loading operation succeeded.

8.3 Machine and User Interface Classes

VoleMachine

Acts as the main container for the machine components: CPU, memory, screen, and program loader. It provides operations to load a program, run the machine continuously or one step at a time, check whether execution has halted, reset the machine, and access its main components.

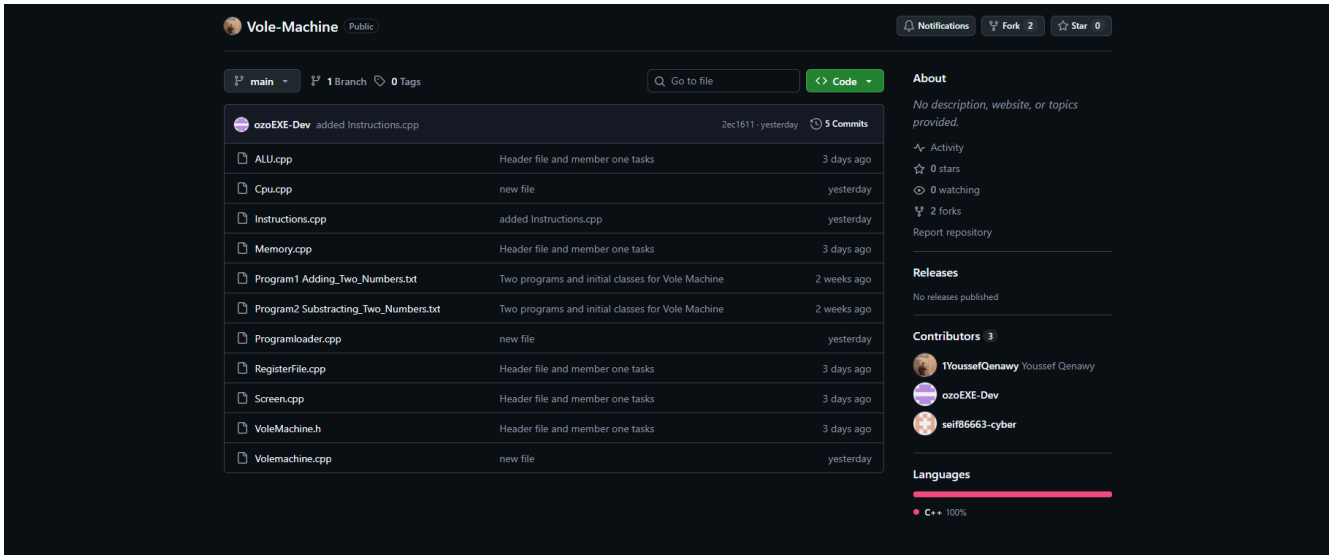
MachineDisplay

Handles displaying the state of the machine. It has separate methods for showing memory, registers, CPU information, and screen output, as well as a method to show the complete machine state.

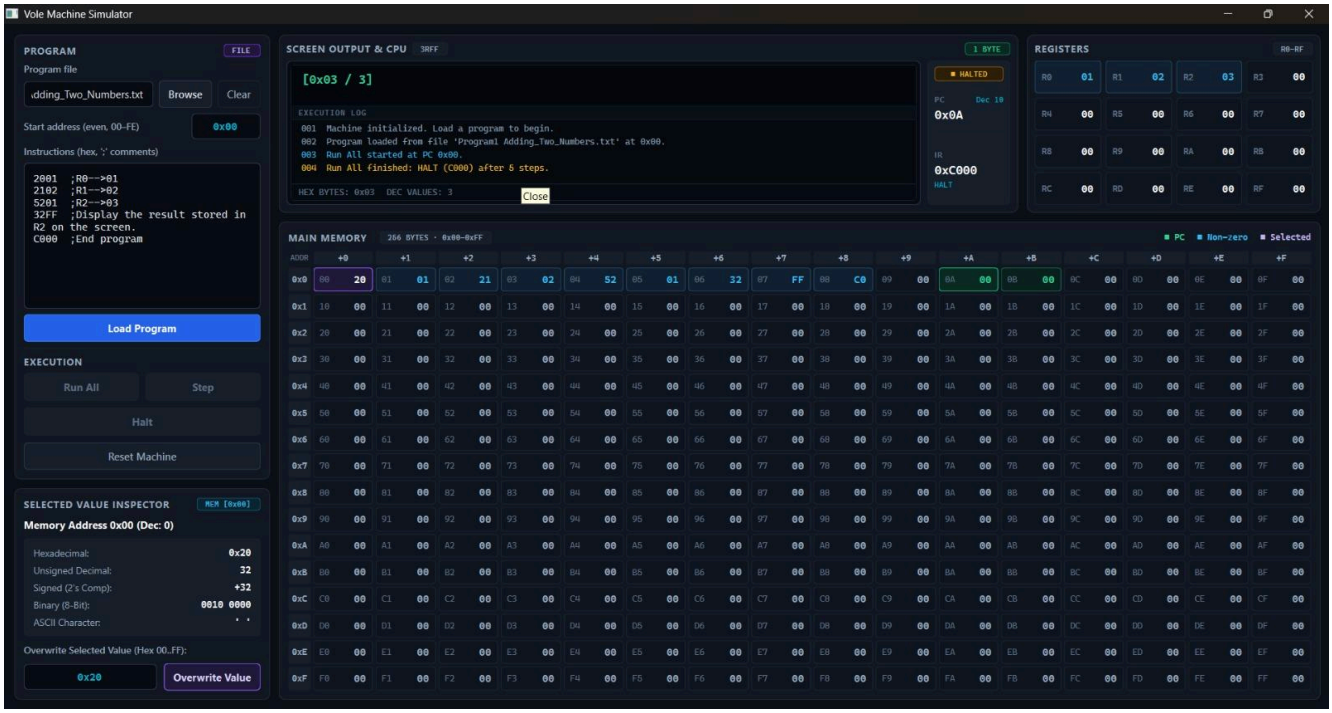
ConsoleUI

Provides the console-based user interface. It owns a VoleMachine and a MachineDisplay, and its start method is the entry point for the console interaction.

9. GitHub Repository



10. User Interface Demonstration



11. Video Links and Outline

a. This video provides a brief overview of the classes declared in VoleMachine.h. The descriptions are based on each class's declared data members, methods, and names.

phase2.mp4

Design Overview Video Outline

- 0:00–1:00 — Introduce the Vole Machine and the goal of the design task.
- 1:00–3:00 — Present the ChatGPT design and its main strengths/limitations.
- 3:00–5:00 — Present the Gemini design and its main strengths/limitations.
- 5:00–7:00 — Present the Claude design and its main strengths/limitations.
- 7:00–9:00 — Compare the three designs and explain which one was selected, using concrete examples.
- 9:00–11:00 — Explain the refinement process and summarize lessons learned.

b. This video provides an overview of the implemented user interface to the program with the testing of two sample programs.

Vole-machine.mp4

- 0:00–1:00 — Introduction to the assignment and a briefed look at the design.
- 1:00–3:00 — Testing both programs and outro.